Master Method:

$$T(n) = \begin{cases} c & n=1 \\ aT\left(\frac{n}{b}\right) + cn & n>1 \end{cases}$$

$$T(n) = aT\left(\frac{n}{b}\right) + cn$$

$$= a^K T\left(\frac{n}{b^K}\right) + cn\left(\frac{a^{K-1}}{b^{K-1}} + \cdots - \frac{a}{b} + 1\right)$$

$n = b^K$ $\quad T(n) = cn\left(\frac{a^K}{b^K} + \cdots + \frac{a}{b} + 1\right)$

$K = \log_b n$

Case (i) $a = b \rightarrow T(n) = cn(K+1)$

$= cn(\log_b n + 1)$

$= \Theta(n \log_b n)$

(ii) $a < b \rightarrow T(n) = cn \frac{1-\left(\frac{a}{b}\right)^{K+1}}{1-\left(\frac{a}{b}\right)} < cn \cdot \frac{1}{1-\left(\frac{a}{b}\right)}$

Case (ii) $a < b \longrightarrow T(n) = cn \cdot \frac{\left(\frac{a}{b}\right)^{K+1} - 1}{\left(\frac{a}{b}\right) - 1}$

$= cn \cdot \frac{1-\left(\frac{a}{b}\right)^{K+1}}{1-\frac{a}{b}} < cn \cdot \frac{1}{1-\frac{a}{b}}$

$= cn\,\Theta(1)$

$= \Theta(n)$

log 2 1

Case-3 : $a > b \rightarrow f(n) = cn \cdot \frac{\left(\frac{a}{b}\right)^{k+1} - 1}{\left(\frac{a}{b}\right) - 1}$

$$= cn \cdot \left(\frac{a}{b}\right)^{k} \quad \left[\text{neglecting } \frac{a}{b} - 1\right] \left[k+1 \approx k\right]$$

$$= cn \cdot \Theta\left(\frac{a^{\log n}}{b^{\log n}}\right)$$

$$= cn \cdot \Theta\left(\frac{a^{\log n}}{n}\right)$$

$$= cn \cdot \Theta\left(\frac{n^{\log a}}{n}\right)$$

$$= \Theta(n^{\log a}) \quad (a > b)$$

- ☑ $T(n) = aT\left(\frac{n}{b}\right) + f(n)$; $a \geq 1$, $b \geq 1$

(i) $f(n) = \Theta(n^c)$ where $c < \log_b a \longrightarrow T(n) = \Theta(n^{\log_b a})$

(ii) $f(n) = \Theta(n^c \log^k n)$ where $c = \log_b a \longrightarrow T(n) = \Theta(n^c \log^{k+1} n)$

(iii) $f(n) = \Theta(n^c)$ where $c > \log_b a \longrightarrow T(n) = \Theta(f(n))$

$$T(n) = 8T\left(\frac{n}{2}\right) + 1000n^2$$

(a = 8, b = 2, $f(n)$ = $1000n^2$)

$$\log_b a = \log_2 8 = 3$$

Here $c = 0$ which is less than 3. so, first case. $\Theta(n^3)$

f(

Divide and conquer

① Convex Hull
② Matrix multiplication (Strasen's)] Self study
③ Quick sort
④ Hip sort
Heap → Priority Queue

4E-Day

$T(n) = aT(\frac{n}{b}) + f(n)$

$T(1) = c$

$a \geq 1$, $b \geq 2$, $c \geq 0$

If $f(n) \in \Theta(n^d)$ where $d \geq 0$

$$T(n) = \begin{cases} \Theta(n^d) \longrightarrow a < b^d \\ \Theta(n^d \log n) \longrightarrow a = b^d \\ \Theta(n^{\log_b a}) \longrightarrow a > b^d \end{cases}$$

$T(n) = T(\frac{n}{2}) + \frac{1}{2}n^2 + n$

$a = 1$, $b = 2$, $d = 2$

Hence, $a < b^d$

Complexity $= \Theta(n^d)$

$= \Theta(n^2)$

$T(n) = 2T(\frac{n}{4}) + \sqrt{n} + n^2$

$a = 2$, $b = 4$, $d = \frac{1}{2}$

$a = 2$, $b^d = 2$

$a = b^d$

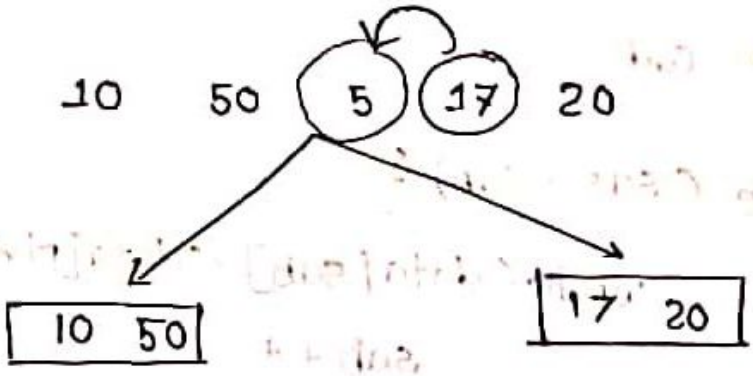
$\therefore$ Complexity $= \Theta(\sqrt{n} \log n)$

Next CT te master theorem included থাকবে।

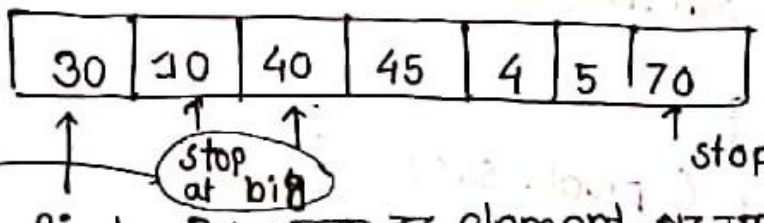
Quick Sort : 10 50 5 17 20

5 10 17 20 50

Through divide conquer



- একজনকে fixed place এ বসাবো Then প্রত্যেক sub-unit এর জন্য recurrence relation call করবো। প্রত্যেকবার কোনো element কে right place এ বসাবো।



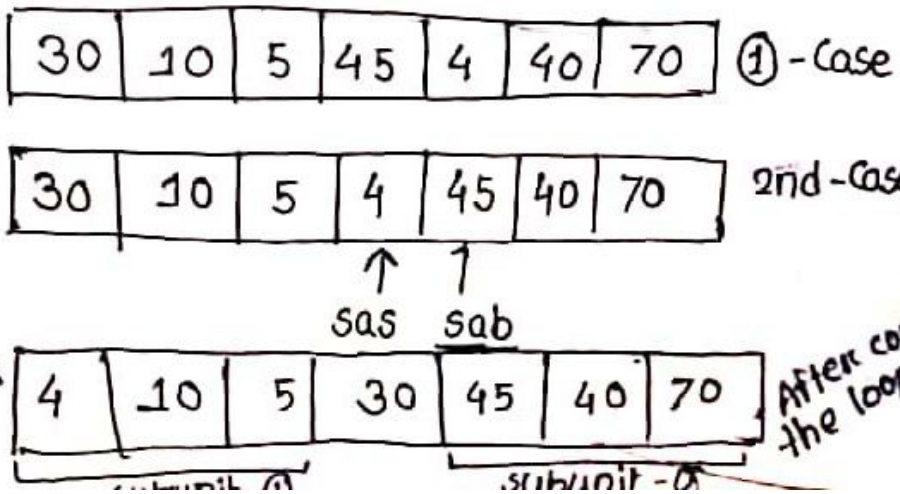
Pivot element } এর চেয়ে বড় element গুলোকে ডানে এবং ছোটগুলোকে বামে আসাবো।

(data[pivot] > data[sab]) ⟶ 40 এ গিয়ে থেমে যাবে
sab++; (ii) 45 " " " "

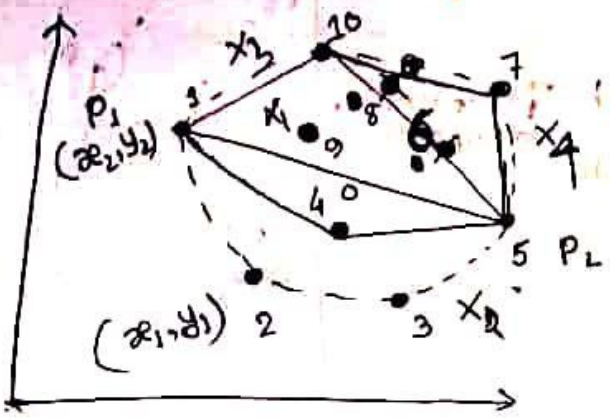
e(data[sas] > data[pivot]) ⟶ 5 এ থেমে যাবে
sas--; (ii) 4 " " "

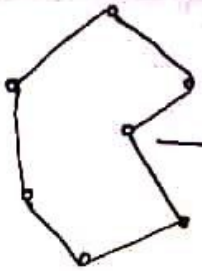
swap(); ⟶ এখানে 40, 5 swap হয়ে যাবে

if(sab ≥ sas)
break;
}
swap(pivot, sas)



Convex hull : पूर्णता





P_1 = find one extreme point (lowest x)

P_9 = find " " " (lowest highest y)

Hull 1 = compute (X_1, P_1, P_2);
P_7, P_{10} // find left hull points of P_1P_2 line
// X_1 is set points of left to $P_1 \vee P_2$

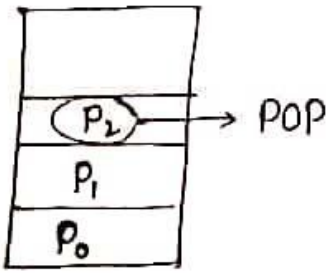
Hull 2 = compute (X_2, P_1, P_2);
P_4, P_3

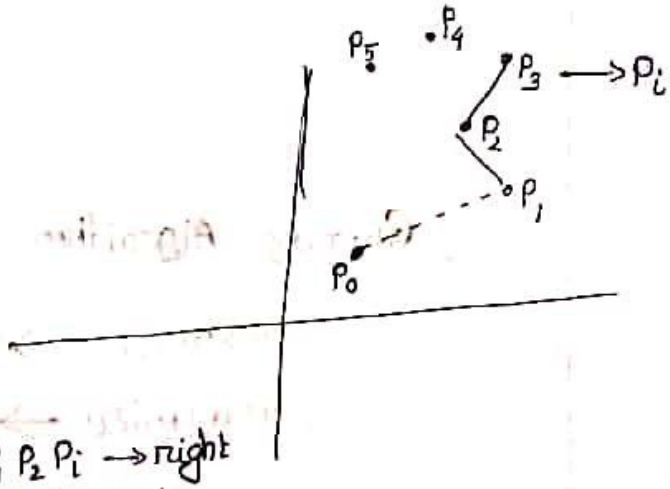
(Hull 1 ( ) Hull 2).

50-Day

ⓐ Convex Hull:

Graham's Scan:





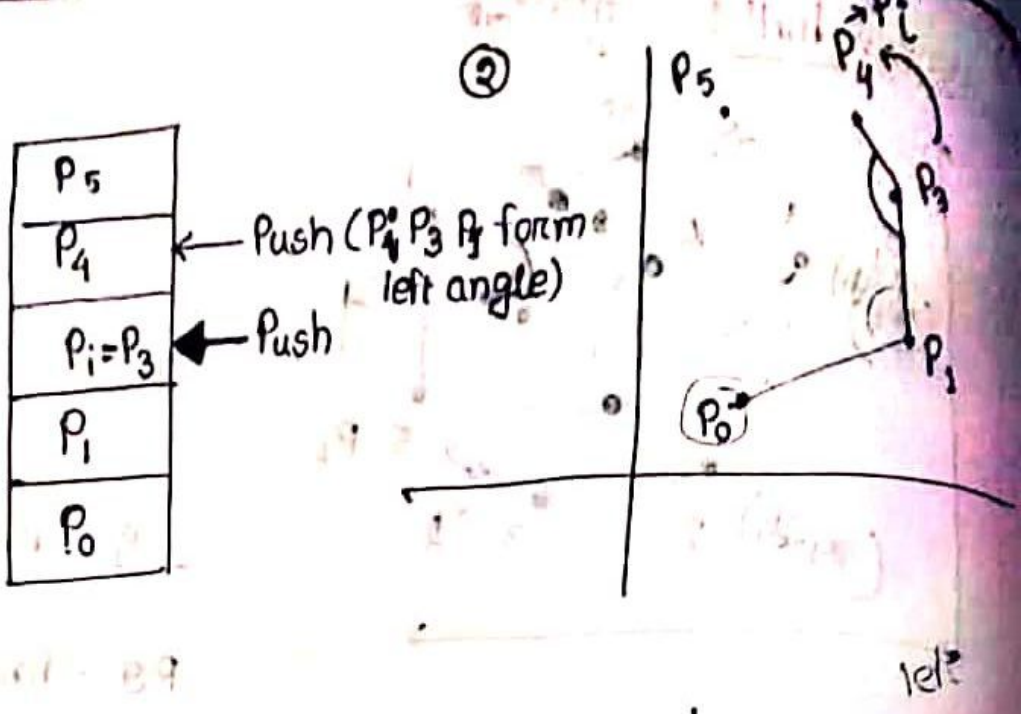
$P_1 P_2 P_i \rightarrow$ right

for(i = 3 to m) angle

while (ⓐ ($P_1 P_2 P_i$ forms right angle) {

multiple pop

Pop();
Push(P_i)



left

⟶ Extreme point search (O(n)) ⟶ linear time

⟶ Sorting of all points (O(nlogn))

⟶ Push ⟶ O(e)

```
for(i=3 to m) {
    while(P1 P2 Pi form non
          angle)
        pop();
    Push(Pi);
}
```

O(m) ←

O(mlogm) ←

O(e)

Greedy Algorithm :

maximize ⟶ profit

minimize ⟶ loss

Problem P

Subproblem P

P_1 P_2 P_3 ..... P_n

S_1 S_2 S_3 ..... S_n

Optimization Problem

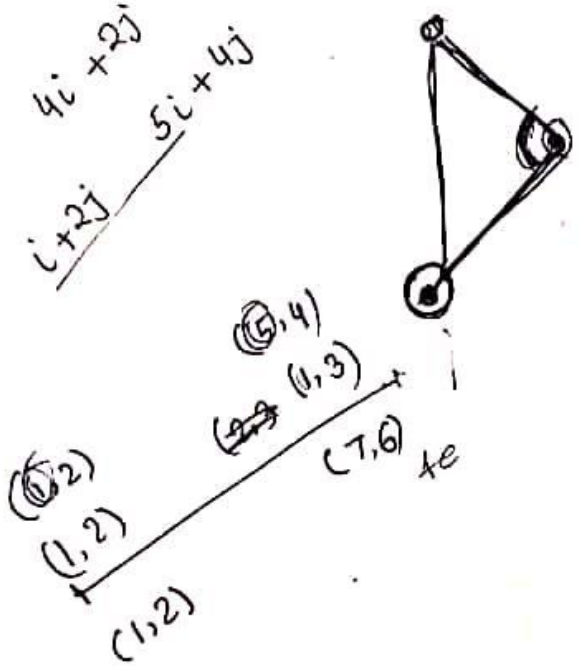
↳ optimal solution select करुपर्व

→ Solution
→ optimal solution
→ Fasible solution
↳ condition satisfy करुपर्व ।

Write a program to initialize

→ Initialize some points to (20)

→ Select the points that form a convex Hull. from these points using Grraham Scam Algorithm



পরবর্তীতে যে দুটো sub unit আসবে তাদের দুকরায় same proc

- Same code

```
while (sas > sab) {
    while (data[sab] < data[pivot)
        sab++
    while (data [sas] > data[pivot)      } n-1
        sas--;
    swap (data[sab], data[sas])
}
swap ( pivot, sas)
```

✓ Deterministic quick sort:
✓ Non-deterministic quick sort:

✓ probabilistic analysis:

✓ Randomize quick sort:

• Time measurement : Lab

2E-Day

Divide and conquer approach:

⟶ split a problem into k subsets

⟶ solve the subsets → iteratively / ~~recussi~~ recursively

After break, when the problem comes to a simple form we will solve it straight forward.

1 20

1 . . . . 10 | 11 20

complexity কমাতে ↑ দুটা part করে ফেলবো

Recurrence Relation:

Example: $T(n) = T(n-1)$

$T(n-1) = T(n-2)$

$T(n-2) = T(n-3)$

Complexity analysis

Master method | Substitution method | Iteration method

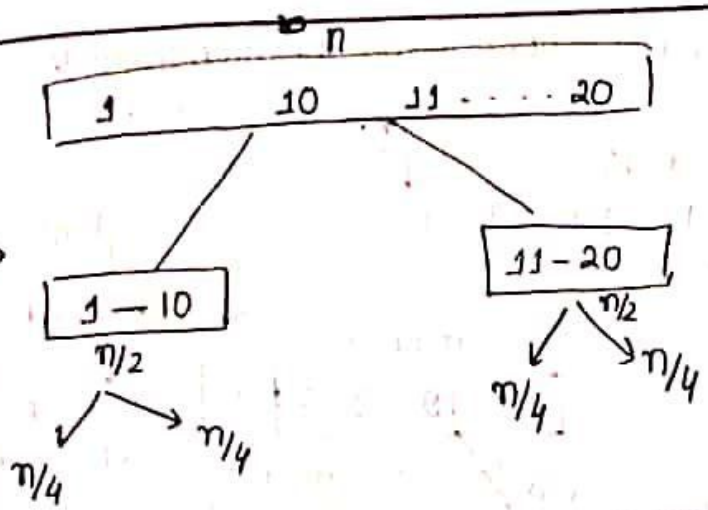
$T(n) = 2T(\frac{n}{2}) + n - 1 \quad \cdots (1) \quad [n>1]$ ← line 1 এর চিহ্নে cost equation এটা

$T(1) = 0 \quad [n=1]$ ← base case

→ কোন ডাটাতে / operation করতে cost fixed হয়ে যায় → এই problem এর ক্ষেত্রে cost zero ধরা হয়েছে

• Master method কে Iteration method থেকে derive করা যায়।

Substitution method: appropiate guess করে complexity calculate করতে হবে and proof করতে হবে। যখন algorithm-এ master method and Iteration method এ used to হয়ে যাবে এবং অহরহই complexity calculate করতে substitution method use হবে।



Complexity of a recurrence relation by iteration method :

$T(1) = 0 \; ; \quad n = 1$

$\rightarrow T(n) = 2T\left(\frac{n}{2}\right) + n - 1 \; ; \; n > 1$

$(i=1)$, $T(n) = (n-1) + 2T\left(\frac{n}{2}\right)$

↓ Iteration 1

$= (n-1) + 2\left\{2T\left(\frac{n}{4}\right) + \left(\frac{n}{2} - 1\right)\right\}$ → আগের step-এ $T\left(\frac{n}{2}\right)$ এর value বসাবে

- second iteration

$i=2 \rightarrow = (n-1) + (n-2) + 4T\left(\frac{n}{4}\right)$ → Iteration-2

$= (n-1) + (n-2) + 4\left\{\left(\frac{n}{4} - 1\right) + 2T\left(\frac{n}{8}\right)\right\}$

$i=3 \rightarrow = (n-1) + (n-2) + (n-4) + 8T\left(\frac{n}{8}\right)$

$\therefore$ ith eque iteration $= (n-1) + (n-2) + \cdots (n - 2^{i-1}) + 2^i T\left(\frac{n}{2^i}\right)$

Assumption: $\frac{2^i = n}{i = \log_2 n}$ [$T(1)$ হওয়ার জন্য $n = 2^i$ হতে হবে]

So, ith iteration becomes $= (n-1) + (n-2) + \cdots (n - 2^{i-1}) + n\,T(1)$

↓ means i amount of n আছে

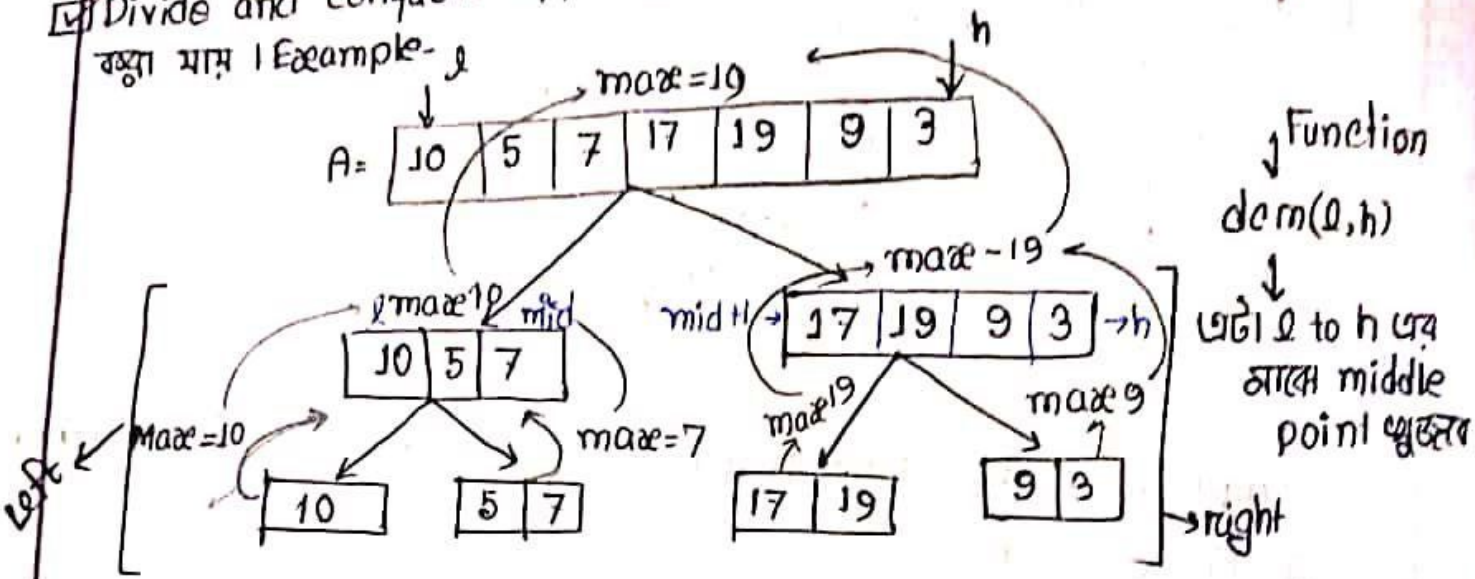
$= (n-1) + (n-2) + \cdots \left(n - 2^i \cdot \frac{1}{2}\right) + nT(1)$

$= (n-1) + (n-2) + \cdots \left(n - \frac{n}{2}\right) + nT(1)$

$= in - (1 + 2 + \cdots \cdots) + 0$

$= n\log n - (1 + 2 + \cdots \cdots) = O(n \log_2 n)$

☑ Divide and conquer approach-এ recurrence relation generate করা যায়। Example-



• base case-এ middle point calculate করে এটাকে দুভাগে ভাগ করা হচ্ছে

Step – ① $mid = \frac{l_1 + l_2}{2}$

Step – ② dem (l, mid); → maxl
dem (mid+1, h); → maxr

Step – ③ if(maxl > maxr)
max = maxl;
else
max = maxr;

☑ dem (l, h) : if(l == h)
max = A[l]
if (l+1 == h)
if A[l] > A[h]
—
else if —
else

আবার ভাঙতে হবে

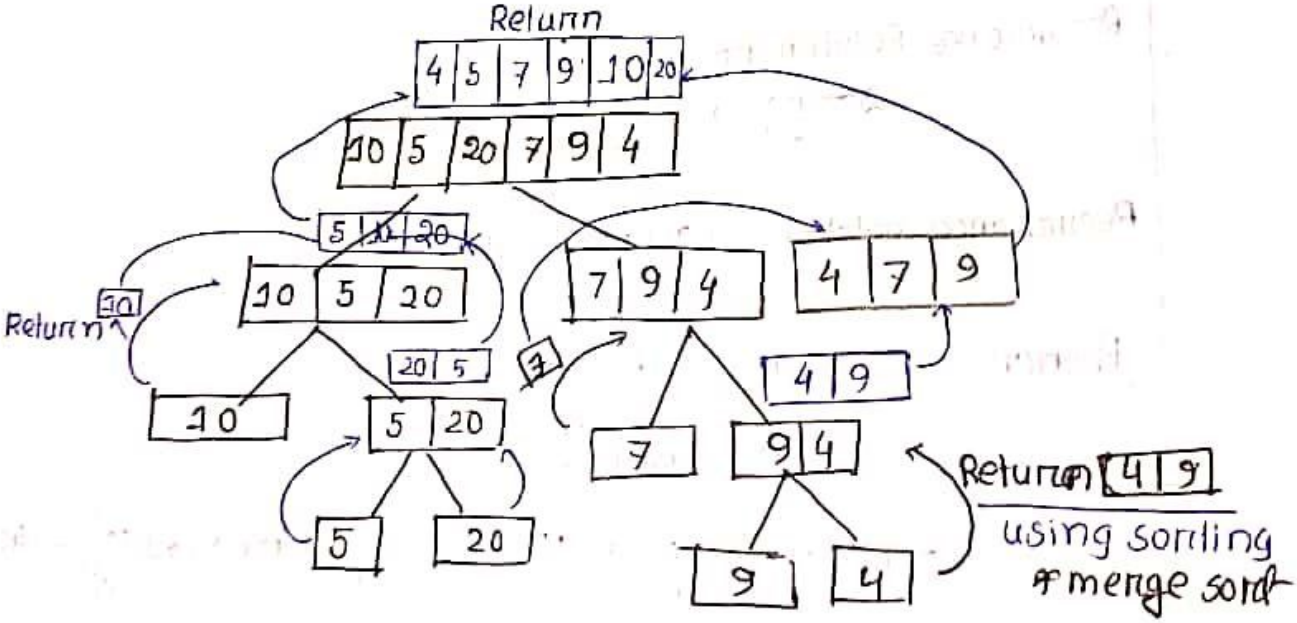
Generating recurrence relation: (max-min finding)

when $n > 2$ $\quad T(n) = 2T(\frac{n}{2}) + 1$ constant cost
এখানে 2 ও 1 দ্বারা বাকি। এখানে cost 1 ধরা হচ্ছে as comparison 1 টা।

$n = 2 \quad T(n) = 1$

$n = 1 \quad T(n) = 0$ → একটা element-এ কোনো comparison নাই

- এগুলো থেকে master/iteration/substion method use করে complexity বের করা যাবে।
- divide conquer approach large amount of element-এ অনেক benefit as complexity n থেকে n/2 হচ্ছে।
- এখন দুটা base case আছে but single node base case হলে complexity হিসাব → self study.



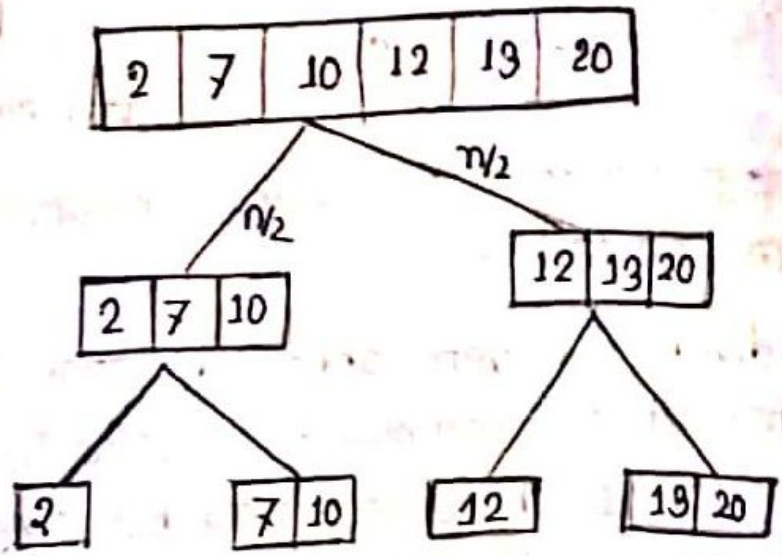
- bubble sort করতে গেলে complexity $O(N^2)$ so প্রত্যেক step-এ N^2 complexity আসবে and ultimate complexity বেড়ে যাবে। So, sort না করে compare করে বড় থেকে ছোটটে merge করবো।

$$T(n) = 2T(\frac{n}{2}) + (n-1) \approx \Theta(n \log_2 n)$$
$$T(1) = 0$$
↳ comparison cost

Total element = n
Total comparison = n

$$T(n) = aT\left(\frac{n}{b}\right) + e + f(n)$$

```
B(l,h,k) : { if(k == A[l])
n>2 {
    mid = (l+h)/2 ;
if (k < A[mid])
Bool isl = BS(l,mid ; k);
se,
Bool isr = BS(mid+1, h, k);
if (isl || isr)
    return true;
}
```



Recurrence Relation for the code:

$2T(\frac{n}{2}) + 1$ → একটাই comparison

Recurrence relation: $T(\frac{n}{2}) + 1 \approx \log n$

Problem : $\underset{a_1}{\underset{6}{123}}\underset{a_0}{\underset{6}{456}} \times \underset{b_1}{\underset{6}{789}}\underset{b_0}{\underset{6}{222}}$

$$= (123 \times 10^3 + 456) \times (789 \times 10^3 + 222)$$

$$= (123 \times 789 \times 10^6) + (123 \times 222)10^3 + (456 \times 789)10^3 + (456 \times 222)$$

$$= (a_1 b_1)10^n + (a_1 b_0)10^{n/2} + (a_0 b_1)10^{n/2} + a_0 b_0 \rightarrow \text{decimal}$$

$$(A \times B)_2 = \underset{c_2}{(a_1 b_1)} \times 2^n + \underset{c_1}{(a_1 b_0 + a_0 b_1)} 2^{n/2} + \underset{c_0}{a_0 b_0} \rightarrow \text{Binary}$$

☐ গুণ করার ক্ষেত্রে গুণ কতগুলো digit এর সাথে করবো সেটা কমানোর try করতে হবে

Total গুণ করতে হচ্ছে 4টা, আর 2^n multiplication

$4(\frac{n}{2})$

$T(n) = 4T(\frac{n}{2}) + \underbrace{f(n)}_{\text{additional যোগ and } 2^n \text{ এর multiplication এর জন্য}}$

$= O(n^2)$

$(a_0+a_1)(b_0+b_1) = a_0b_0 + a_1b_0 + a_0b_1 + a_1b_1$

$(a_0+a_1)(b_0+b_1) - (c_2+c_0) = a_0b_0 + a_0b_1 + a_1b_0 + a_1b_1 - \underline{a_0b_0 - a_1b_1} - (c_2+c_0)$

$(a_0+a_1)(b_0+b_1) - (c_2+c_0) = a_0b_0 + a_0b_1 + a_1b_0$

$(A \times B)_2 = (a_1b_1) \times 2^n + \{(a_0+a_0)(b_1+b_0) - (c_0+c_2)\} \times 2^{n/2} + a_0b_0$

110110 } n
111111 }
↳ যোগের জন্য n size এর -

$T(n) = 3T(\frac{n}{2}) + f(n)$

$= 3T(\frac{n}{2}) + cn$

$= O(n^{1.5\cdots})$

↳ For large number huge improvement হবে

- কোনো number কে 2 দিয়ে গুণ একবার left shift করবে / n বার গুণ n বার shift করবে

MPY(A, B)

$A = [a_1, a_0]$

$B = [b_1, b_0]$

$T(n/2)$ ← $c_0 = a_0b_0$ — MPY(a_0, b_0)

$T(n/2)$ ← $c_2 = a_1b_1$ — MPY(a_1, b_1)

$T(n/2)$ ← c_T : MPY($(a_0+b_0)(b_1+b_0)$)

$c_1 = c_T - (c_0 + c_2)$

return = $c_0 + c_1 \times 2^{n/2} + c_2 2^n$

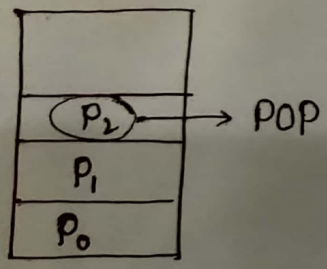
P_4, P_3

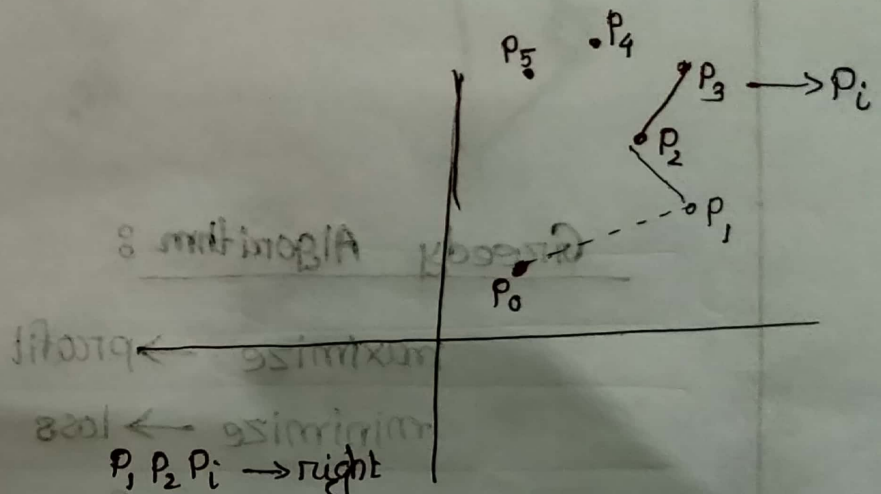
(Hull 1 ( ) Hull 2)

5D-Day

ⓐ Convex Hull:

Graham's Scan:





$P_1\ P_2\ P_i$ → right

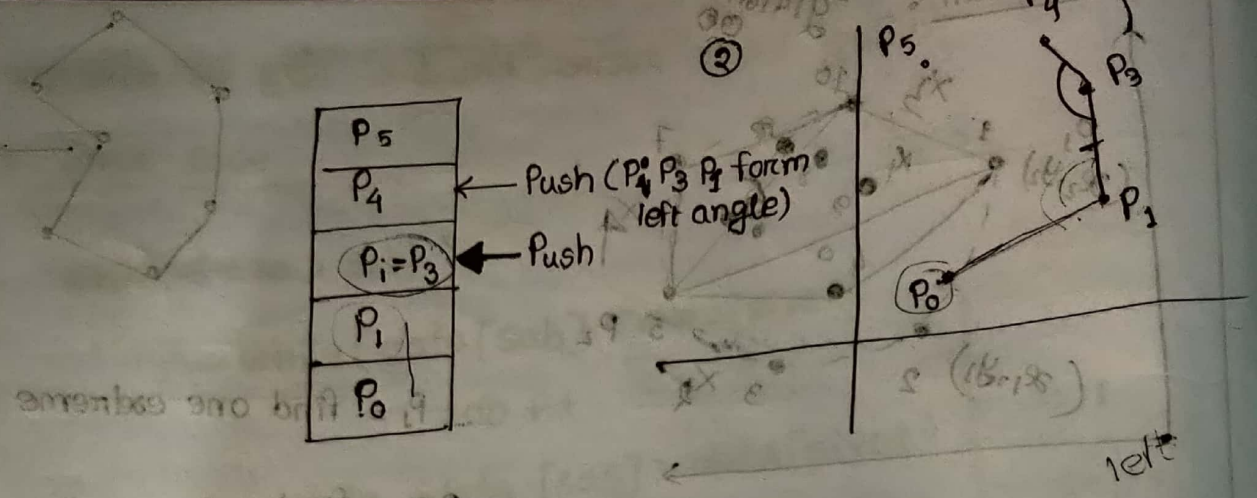
for(i = 3 to m) angle

while (Ⓐ ($P_1\ P_2\ P_i$ forms right angle) {

multiple pop

Pop( );
Push(P_i)

3



→ Extreme point search (O(n)) → linear time

→ Sorting of all points (O(nlogn))

→ Push → O(c)

O(m)
O(mlogm)
O(c)

```
for(i=3 to m) {
    while(P1 P2 Pi form nonleft angle)
        POP();
    Push(Pi);
}
```

Greedy Algorithm :

maximize → profit

minimize → loss

Problem P

Subproblem P	P_1	P_2	P_3	.....	P_n
	S_1	S_2	S_3	.....	S_n

Optimization Problem

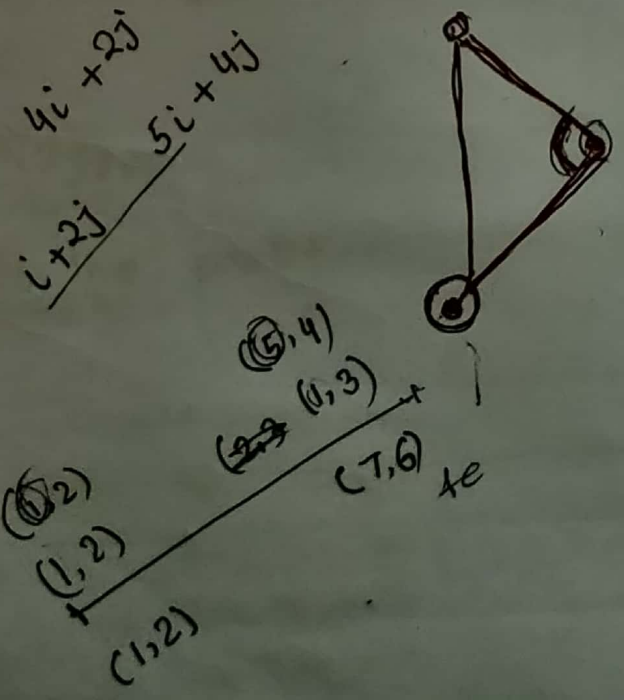
↳ optimal solution select करूरत

→ Solution
→ optimal solution
→ Fasible solution
↳ condition satisfy करूरत ।

Write a program to initialize

→ Initialize some points to (2D)

→ Select the points that form a convex Hull. from these points using Graham Scan Algorithm



Convex Hull (Graham's Scan)

→ Select the base point which has lower y-value.

→ Sort all the remaining points in order of their polar angle from P_0.

→ Initialize a stack, S.

→ push (S, P_0)

→ Push(S, P_1)

→ Push(S, P_2)

→ for (i=3 to n) do

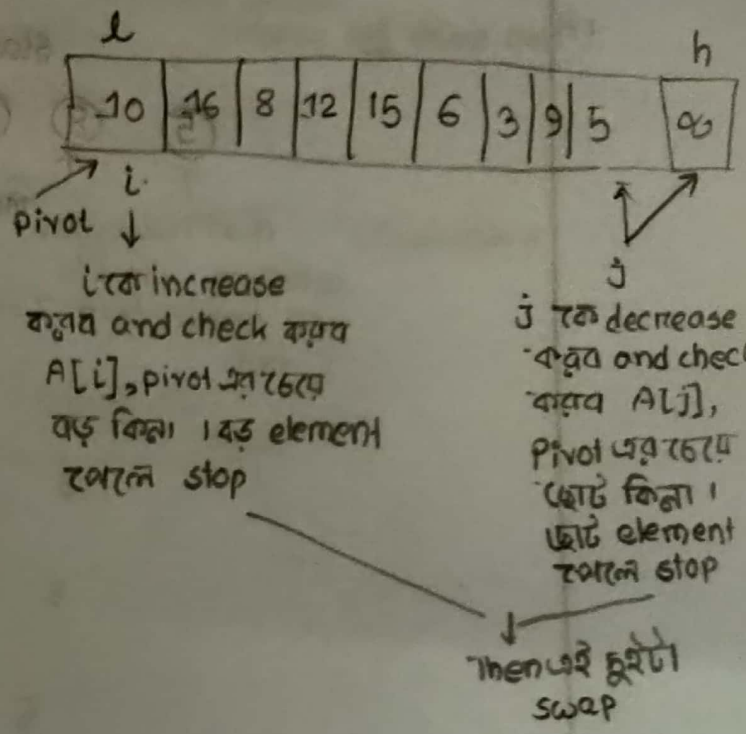
while (angle formed by topnext(S), top(S) and P_i make s right turn)

- pop(S)
- push(S, P_i)

→ return S.

Quick Sort : pivot = first element

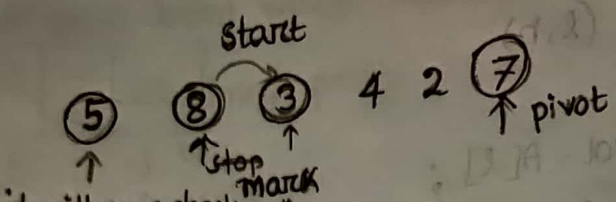
$O(n^2)$

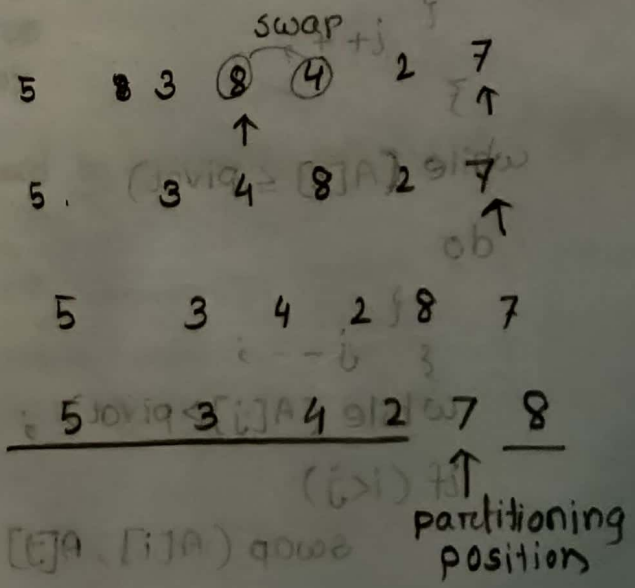



```
Partition (l,h)
{
    pivot = A[l] ;
    i = l ; j = h;
while(i<j) {
    do
      { i++ ;
      }
    while (A[i] ≤ pivot)
    do
      {
    } j-- ;
    while (A[j]>pivot) ;
    if (i<j)
        swap (A[i], A[j])
  }
  swap (A[l], A[j]);
  return j ;
      ↳ This is the partitioning position)
}

Quicksort (l, h)
{   if (l < h) {
      j = partition (l,h) ;
      Quicksort (l, j);
      Quicksort (j+1, h) ;
}
```

Quick sort → last element pivot



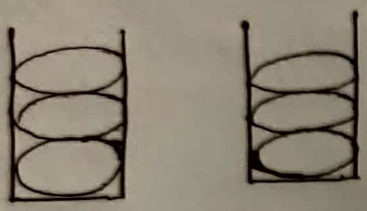


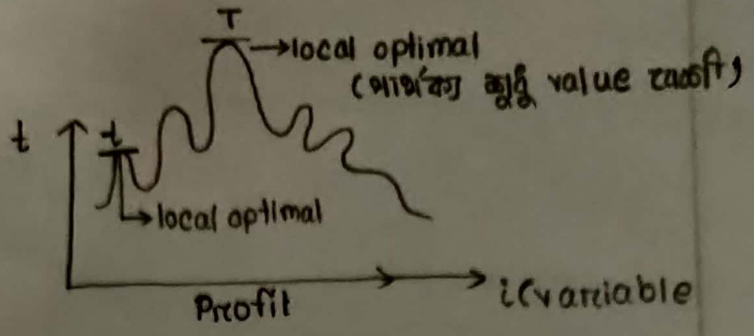
* Quick Sort (Randomize)

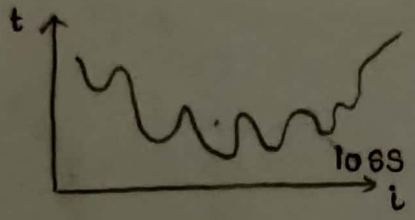
low point
high point

```
Randomized QS(A,p,r){
if (p<r) {
    Randomly selected pivot element
    i ← random(p,r);
    swap(A[i], A[r]);
    q ← partition(A,p,r);
    Randomized QS(A,p,q-1);
    Randomized QS(A,q+1,r);
}
```

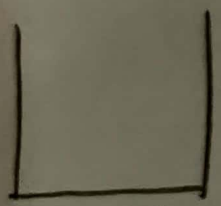
5E-Day







Q1 knapsack



knapsack
capacity = K(Kg) = 37kg(Let)

object	0	1	2	3	4	5
ω_i	10	5	15	20	8	7
P_i	5	10	15	7	8	12
Profit						

feasible solution :

$(\Sigma \omega_i x_i) \leq K \mid x_i \in [0, 1]$

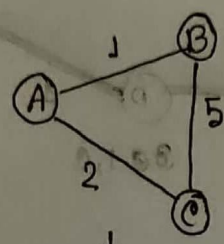
x_i এর object কে নিব না

i তম object কে নিব

$f = (\Sigma P_i x_i)$

$= \max (\Sigma P_i x_i)$ max ← $\Sigma \omega_i$, Σx_i, ΣP_i

- Dijkstra's Single source shortest Path
- Prim's Algorithm ⟶ Minimum Spanning tree

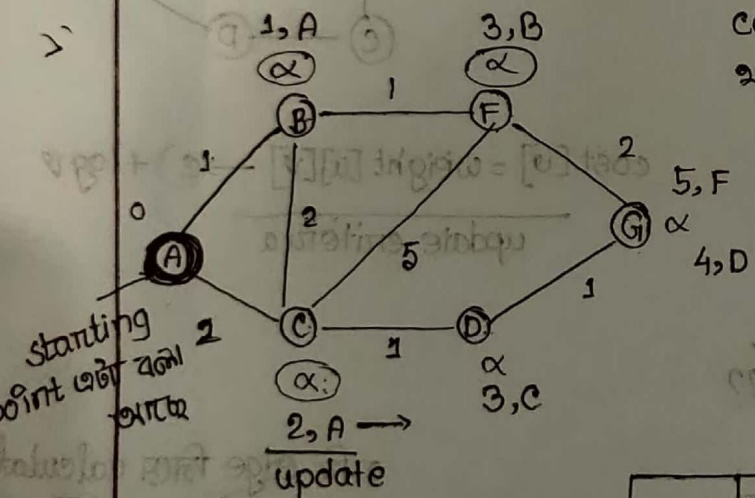


5 ⟶ এটা বাদ দিব

V ⟶ nodes
E ⟶ Edges

↓

এখান থেকে একটা edge বাদ দিয়ে minimum spanning tree বানানো যায় যার cost কম হবে and সবগুলো node এর মধ্যে থাকবে।



starting point এটা বলা আছে

Source code sample

```
set all cost of all nodes = ∝;
cost[source] = 0
push all nodes into PQ
while datastructure is not empty {
    u = GetMin ( )
    for all v ∈ adj[u]
        if ( ) → update(u,v)
        update criteria
}
```

u = GetMin ( ) ↓ যার cost minimum সেই node টা দিচ্ছে

v সংখ্যক বার চলবে

Queue তে আছে কিনা

Node	Cost
A	0
B	∝
C	∝
D	∝
F	∝
G	∝

Step-1

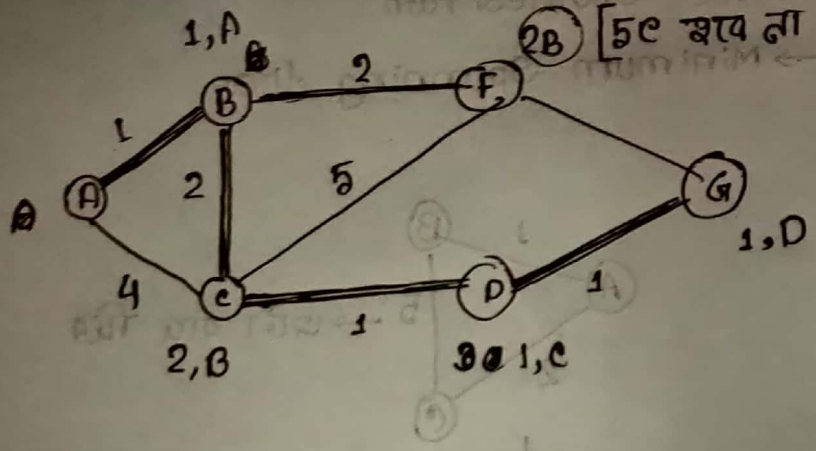
So, u = A ⟶ mincost

then A এর adjacent গুলোকে update করা হলো।

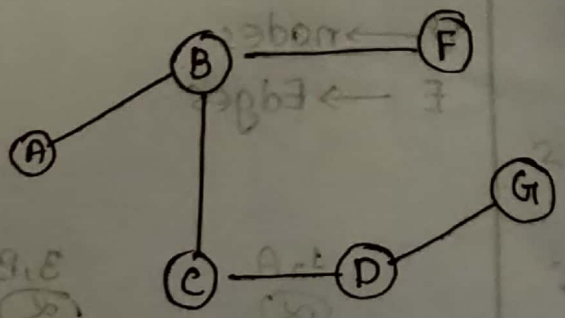
Step-2

So, u = B (as mincost . B এর adjacent C, F। C আগে থেকেই updated So, C update হবে না। F update হবে only F = 3, B

Minimum spanning tree source থেকে cost count করবে না



minimum spanning tree



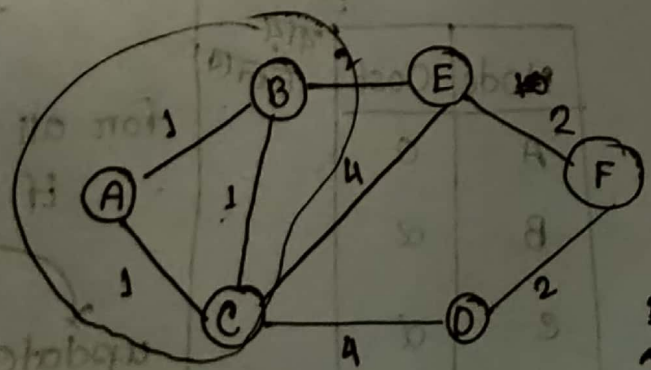
Sample source code

```
u = GetMin( )
for all v ∈ adj[u]
if (                )
    update (u, v)
```

update criteria ? ?

cost[v] = weight[u][v] → (c) + log v

update criteria



যদি edge নিয়ে calculate করে

AB

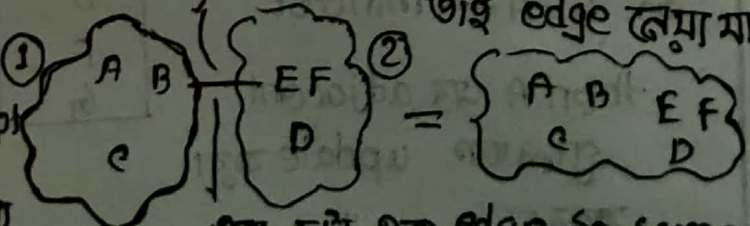
BC

CA এমন কিছু আসবে।

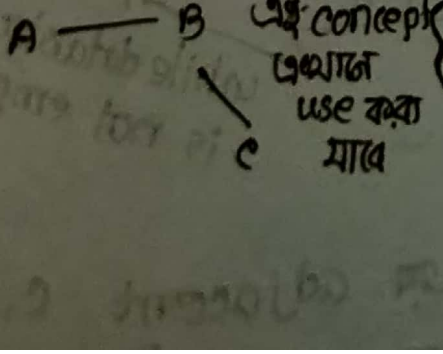
So, minimum spanning tree আর থাকবে না।

তাই edge নেয়া যাবে না

Disjoint Set

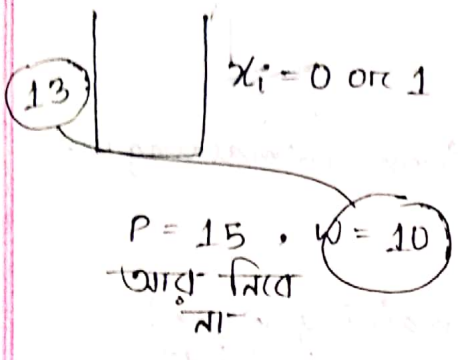


এরা দুটা এক edge so same group এ থাকার কথা তাই group দুটো same করে দিবে।



02/03/2020

0/1 knapsack



brute force
greedy

same - 3 আসতে পারে
greedy তে soln কম 3 আসতে পারে, এক্ষেত্রে, greedy reliable না cz, brute force-এ 3 soln বেশি আসতে পারে.

Overlapping Soln:

i	w_i	P_i
1	1	2
2	2	3
3	3	4
4	4	5

Max Profit = ?

Fractional knapsack

i	w_i	P_i → Greedy
1	10	15
2	5	✗

greedy reliable

W = 10+3
$P = 15 + \frac{\times}{5} \times 3$

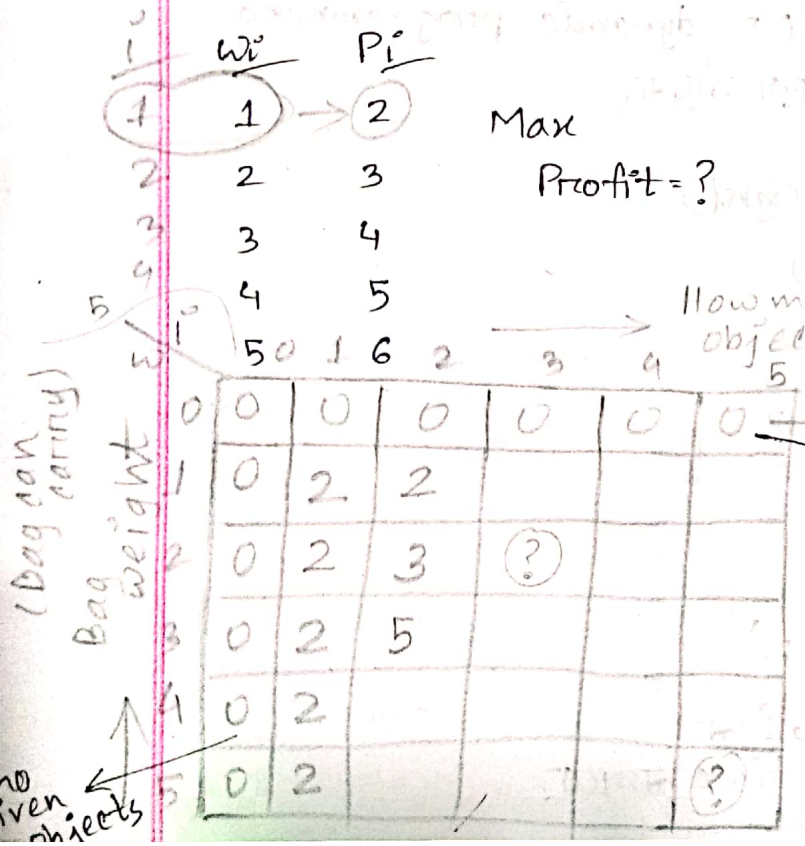
greedy limitations:

weight greedy	w	P_i
হলে,	10	17 ✗
weight wise sort	5	15

করে নিলে, 10 আগে 10 pack করে 5-এ আসলে, profit বেশি দেখে আবার 10 কে বাদ দিয়ে 5 নেওয়ার way নাই, profit greedy তে back করবেনা. parameter একটাই থাকবে.

ONE WAY

প্রতি step-এ যাকে pack করা হচ্ছে তাকে আবার process করা যায়না.



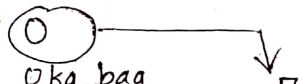
column-1 object will be included if, $w_i \leq w$
MP[w,i] = —

Column-2 Profit from 2nd obj + Pr. from 1st obj.
(2,1) = 0 + 2

2nd column-এ, আগের column থেকে profit নেয়া যাবে, same profit 2 বার solve করতে হবেনা। এই কাজটা greedy তে করা যায়না।

dividing into sub-problem,

profit from 2nd obj + profit from remaining obj

$(2,1) \equiv 3 +$ 

এরকম soln আগের column-এ খুঁজব

$(0,1) \rightarrow 0$

MP [RO, RW]

$(3,3) \equiv 3 + 2$

P[i]

MP[W,i]

P[i] + MP [RO, RW]

RW = W[i] − w[i]

RO = i − 1

- subproblem reuse করাকে dynamic programming বলা হয় যা greedy তে করা যায়না।
- for (w = 0 ⟶ weight)
  for (i = 0 ⟶ n)
  { ——————
  MP[w,i] = ?
  } ——————

worst case O(W × n)

just 1টা index চাইলে- তখন constant time লাগবে।

- Halfman coding
- AVL tree

Write a program to implement,

- kruskal's MST algorithm on a connected undirected graph
- 0/1 knapsack problem using dynamic programming.

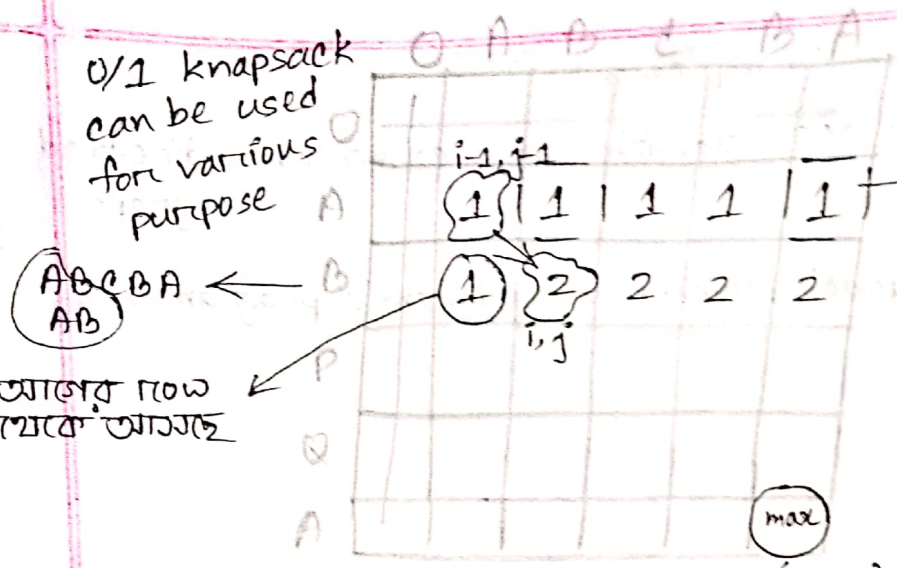
100 20
50 10
150 30

limit = 50 (wt)

ans = 250

08/03/2021

0/1 knapsack can be used for various purpose



ABCBA
AB ← আগের row থেকে আসছে

ABCBA
A → আগের length = 1 A যা যেকোন একটা মেলানো যাবে.

~~ABCBA~~
A
last-এ match করলে 1st টা count হবেনা.

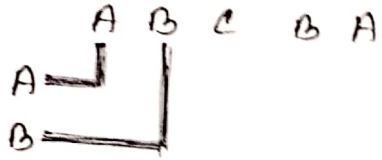
Longest Common Subsequence (LCS):

দুটা string এর মধ্যে সবচেয়ে লম্বা common part বের করা

→ ABCEFBA
ABPQA

ABCBA
AB

Considers: ABCBA = X
ABPQA = Y



এভাবে match করে.

if (Y[i] == X[i])
LCS[i][i] = (++)

else
LCS[i][i] = max (LCS[i][j-1]; LCS[i-1][j])

LCS[i-1][j-1]+1 } part থেকে আসবে

LCS[i][j-1] আগের column থেকে

LCS[i-1][j] আগের column " row থেকে

2 ভাবে আসতে পারে match for AB then, এদের মধ্যে max টা নিব.

09/03/2020

AABACDAB ⟶ compression

Bit = 8×3 ⟶ 5 unique character
= 24

unique character 4টা so, null সহ character 5টা.
∴ 3 bit লাগবে represent করার জন্য.

A = 000
B = 001
C = 010
D = 011
E = 111

24

Encoding fixed length

0 = A
01 = B
010 = C
011 = D
110 = E

4+4+3+3 = 14

∴

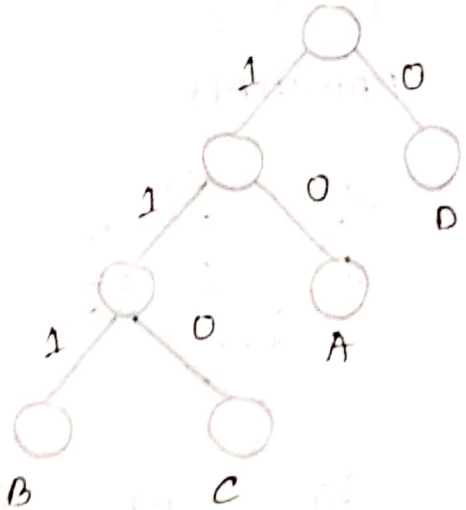
এভাবে fixed length-এও, variable length-এও represent করা যেতে পারে.

24, 14 ⟶ variable length-এ compressed হয়ে থাকে অনেকখানি, কিন্তু এক্ষেত্রে bit representation important.

Prefix Code:

~~fixed~~

A = 0 —— 0
B = 1 —— 1
C = 100 —— 100
D = 011 —— 011



যাদের frequency কম, কম সংখ্যকবার use হবে তাদের tree-র নিচের দিকে থাকবে.

AABACDAB

0 0 1 0 1 0 0 0 1 1 0 1
A B C A D B

decode করার সময়,

0 1 0 0 1 0 — C C
A A A

একটার part হিসেবে যদি আরেকটা চলে আসে তখন সেটা prefix.
এজন্য code unique হতে হবে.

A=0
B=01
C=010

C হতে B, A চলে আসায় so, unique হচ্ছে না.

change,
C = 100

100 100
decode করলে just C-ই আসবে.

A = 0
B = 1
C = 100
D = 011

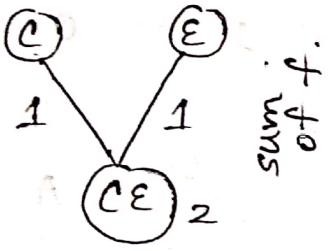
} code generation unique হতে হবে।

BACADAB
10100011101
BABAAAABBAB
misleading in decode

A → 3
B → 1
C → 1
D → 2
E → 1

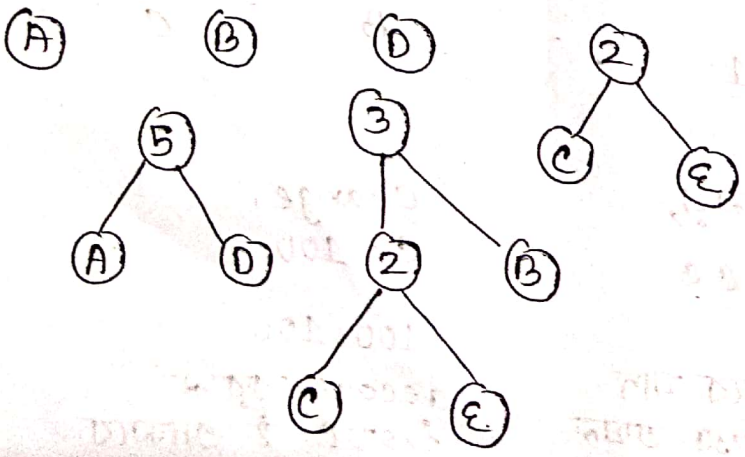
} frequency

AABCDADE



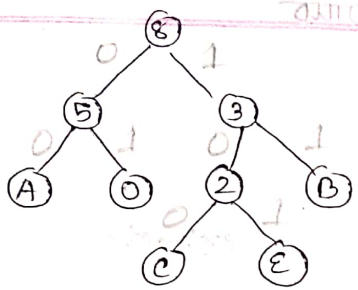
Algo:
1. select two lower frequency node (subtree).
→ Merge them into a tree/n
→ set the frequency of Ne node/tree as the sum of child node.

প্রতিবার lowest frequency-র 2টা node নিয়ে যোগ করব, new node-এর হবে আগের 2টার sum.



update-1

update-2



lowest frequency নিচে থাকবে

update-3

then, encode-decode করা যাবে.

- new node কে আগের data structure-এ ঢুকাতে হবে.

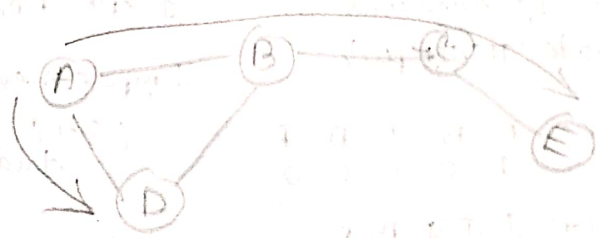
- priority queue এর priority হল frequency.

- Huffman tree
- এ পদ্ধতি encode করা হল, Huffman code
- প্রতি step যাবে, ask করবে leaf node কিনা, প্রতিবার update হলে character গুলো leaf node-এ চলে যাবে.

∴ character leaf node-এ থাকে.

- প্রতিদিন msg change হয় so, frequency-ও change হয়, এজন্য, probabilistic calculation করা হয় fixed value না রেখে যে frequency কার কেমন হতে পারে 1 hour / 1 day পর.

DFS: Depth first search
stack



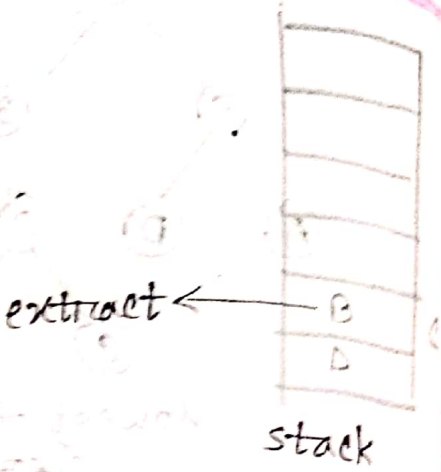
Algo

- extract a node
  → and process/ print
- (put) push its adjacents
  → into (stack / queue)
  DFS BFS

- BFS: queue according to data-structure

- DFS যে রাস্তায় নামবে সেই রাস্তায় যতদূর যাওয়া যায় depth পর্যন্ত complete করবে.

DFS → searching algorithm



Output

A
B
C
E
D

একবার stack-এ আসছে মানে process done.

BFS → Queue use হবে stack এর পরিবর্তে.

print এর order change হয়ে যাবে DFS থেকে.

push() pop()
enq() dq()

adjacency matrix দিয়ে adjacent node পাওয়া যাবে-

একবার push করা হলে তাকে mark করে রাখতে হবে যাতে একাধিক node-এর adjacent হলে same node বারবার push না হয়.

marked[5] = A B C D E
1 0 0 0 0

visited হলে - 1 করে দিব marked array তে.

- find out if any graph has cycle or not?
- directed acyclic graph কিনা?

Implement (Self study)

- acyclic - cycle নাই

Back-tracking - এ DFS লাগবে.